

DepWareTrans: Dependency-Aware Incremental Repository Migration across Co-executable Languages

Sivajeet Chand[✉]
TU Munich
Munich, Germany
sivajeet.chand@tum.de

Alexander Pretschner
TU Munich
Munich, Germany
alexander.pretschner@tum.de

Steve Haupt
andrena objects
Munich, Germany
steve.haupt@andrena.de

Derui Zhu
TU Munich
Munich, Germany
derui.zhu@tum.de

Sushant Kumar Pandey
University of Groningen
Groningen, Netherlands
s.k.pandey@rug.nl

Abstract

Repository-level code translation is critical for modernizing legacy systems, yet existing approaches based on large language models (LLMs) operate at the file level and fail to scale to codebases with complex inter-file dependencies. This limitation is evident in our industrial setting, where we aim to migrate a production repository (STAR) from Java to Kotlin, but file-level approaches produce fragmented results and fail to achieve end-to-end correctness. In this paper, we show that the primary cause of failure at the repository level is dependency inconsistency. Through an empirical study on open-source and industrial systems, we find that most errors arise from unresolved cross-file dependencies that cannot be effectively addressed by iterative feedback alone. We propose a dependency-aware incremental migration framework that elevates the unit of translation from individual files to dependency-consistent batches. Our approach constructs a dependency graph, groups interdependent files, and performs batched translation with iterative compile- and test-driven validation. We evaluate our method on a 51K line of code (LOC) industrial system and multiple repositories across interoperable language pairs (Java–Kotlin, Java–Scala, and C#–F#). On the STAR repository, file-level approaches achieve 38.16% compilation and 9.39% test success, whereas our approach achieves 100% compilation and test success across the evaluated settings, converging within a small number of iterations. These results show that dependency-aware batching improves scalability and reliability in repository-level code translation.

CCS Concepts

• **Software and its engineering** → **Software maintenance tools**; **Maintaining software**; **Software evolution**.

Keywords

Repository, Dependency-Aware, Code, Translation, Migration

ACM Reference Format:

Sivajeet Chand, Alexander Pretschner, Steve Haupt, Derui Zhu, and Sushant Kumar Pandey. 2026. DepWareTrans: Dependency-Aware Incremental Repository Migration across Co-executable Languages. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3832783.3834505>

1 Introduction

Migrating software systems across programming languages is a recurring and high-impact challenge in industrial software development. Organizations undertake such migrations to adopt modern language features, improve maintainability, and align with evolving ecosystems. Large-scale efforts reported by industry, including migrations from Java to Kotlin in Android and enterprise systems, demonstrate both the practical importance and engineering difficulty of these transformations [1, 18, 19].

A widely adopted strategy in practice is *incremental migration*, where systems are gradually transformed while remaining continuously buildable and testable. This approach is particularly feasible for *interoperable language pairs*, i.e., languages that share a common runtime and can coexist within the same system (e.g., Java–Kotlin on the JVM, Java–Scala, and C#–F# on the .NET platform). Interoperability enables mixed-language systems, allowing organizations to migrate large repositories without requiring disruptive, full-system rewrites [11]. However, while interoperability makes incremental migration feasible, it does not make it easy. Instead, it introduces a distinct challenge: maintaining consistency across interdependent components written in different languages. In practice, migration is often performed at the level of individual files or small code fragments, using IDE-based converters or large language model (LLM)-assisted translation tools [15]. Although effective for small or loosely coupled systems, such file-level workflows do not scale to large, dependency-rich repositories. In particular, translating files in isolation frequently breaks dependency relationships across components. This issue is exacerbated in real-world systems with tightly coupled or cyclic dependencies, where interdependent files must evolve consistently. As a result, even when individual translations are locally correct, the system as a whole often fails to compile or pass tests. This creates significant integration overhead and limits the practical effectiveness of existing migration workflows.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3834505>

Recent research has begun to explore code translation using LLMs [10, 16, 25, 28, 31]. At repository-scale, approaches such as Syzygy [26] and Oxidizer [32] decompose programs into fine-grained units (e.g., functions) and validate translations using I/O equivalence, while AlphaTrans [13] introduces dependency-aware ordering over fragment-level units. While these methods improve scalability, they continue to treat translation as a collection of independent units, using dependencies primarily as ordering constraints. Interdependent components are still translated in isolation, leading to integration inconsistencies in practice. This limitation is further highlighted by RepoTransBench [27], which shows that strong performance on fine-grained translation does not transfer to repository-level settings, where inter-file dependencies and system-level validation become critical.

In this work, our approach is motivated by a real industrial migration effort involving a large Java-based repository (STAR, $\approx 51K$ LOC). STAR is an internal time-tracking and project management web application (400 daily users) with a large, extensively tested codebase. The goal of this migration is to adopt Kotlin to benefit from improved type safety (e.g., null-safety), conciseness, and modern language features, while preserving compatibility with the existing JVM-based ecosystem.

In practice, we initially followed a standard file-by-file migration workflow, supported by automated translation tools and iterative compile- and test-driven feedback. However, this approach quickly proved infeasible at scale. Despite repeated refinement, the system frequently failed to compile or pass tests due to inconsistencies across interdependent files. Importantly, these failures were not caused by local translation errors, but by unresolved dependencies and incompatible intermediate states introduced during incremental migration. These observations expose a fundamental limitation of existing workflows: translating files in isolation does not preserve dependency consistency in large, tightly coupled repositories. As a result, migration becomes unstable, requiring significant manual effort to resolve integration issues. This suggests that repository-level migration should be treated as a dependency consistency problem, rather than a collection of independent translation tasks.

We propose **DepWareTrans**, a dependency aware migration framework that aligns translation units with the repository's dependency structure. Specifically, our approach groups interdependent files into dependency-consistent batches (i.e., subgraphs of the dependency graph) and performs joint translation of each batch using iterative compile- and test-driven validation. Unlike prior work, we preserve the original test suite and leverage language interoperability to validate correctness throughout the migration process, including in the presence of cyclic dependencies. We evaluate our approach on the industrial repository and multiple open-source systems. Our results show that while file-level baselines achieve reasonable performance on small repositories, they fail to produce a compilable system in large, dependency-rich settings. In contrast, DepWareTrans achieves end-to-end migration with full compilation and test-suite correctness. We further demonstrate that the approach generalizes across interoperable language pairs, including Java–Scala and C#–F#.

To guide our study, we address the following research questions:

RQ1: To what extent do existing file-level and iterative translation workflows satisfy repository-level migration requirements?

RQ2: How effective is dependency-aware batching in improving correctness and scalability?

RQ3: Does the proposed approach generalize across interoperable language pairs?

In summary, this paper makes the following contributions:

- **Empirical analysis:** We show that file-level migration fails to scale to large, dependency-rich repositories, as most failures arise from dependency inconsistencies rather than translation errors.
- **Methodology:** We propose a dependency-aware batching framework for repository-level migration between interoperable language pairs, enabling joint translation of interdependent components with iterative validation.
- **Practical validation:** We demonstrate the effectiveness of our approach on a large industrial system and multiple open-source repositories.
- **Generalizability:** We show that the approach extends beyond Java–Kotlin to other interoperable language pairs.

2 Related Work

Recent work has explored the use of LLMs for code translation [10, 16, 25, 28, 31], achieving strong performance on curated benchmarks but exhibiting significant degradation on large, real-world repositories. Repository-scale approaches such as Syzygy [26] and Oxidizer [32] decompose programs into fine-grained function-level units and validate translations using I/O equivalence. AlphaTrans [13] translates fragment-level nodes ordered by dependency graphs and relies on translated test suites. Other methods leverage transpilers [29] or generated specifications [22] to guide translation, but depend on the availability of language-specific tooling or auxiliary artifacts. Recent benchmarks such as RepoTransBench [27] further highlight that repository-level translation remains an open challenge, with low success rates even for state-of-the-art methods. A common limitation across these approaches is the reliance on fragment-level decomposition, where translation units correspond to nodes of the dependency graph and dependencies are handled implicitly via ordering or local validation. This fails to ensure dependency consistency within translation units and often leads to integration errors and ambiguous correctness signals. In contrast, our approach constructs dependency-consistent batches corresponding to subgraphs of the repository dependency graph, enabling coordinated translation of interdependent components and validation against the original test suite for unambiguous correctness.

Prior work on interoperable language pairs such as Java–Kotlin, Java–Scala, and C#–F# has primarily focused on empirical studies of migration and cross-language interaction [11, 19, 20], showing that real-world migration is incremental and involves tightly coupled dependencies in mixed-language repositories. However, these works do not propose automated repository-scale migration techniques or address how to structure translation units under such dependency constraints. Our work bridges this gap by explicitly leveraging dependency structure to guide migration and by demonstrating its effectiveness not only on open-source benchmark repositories but also on an industrial codebase.

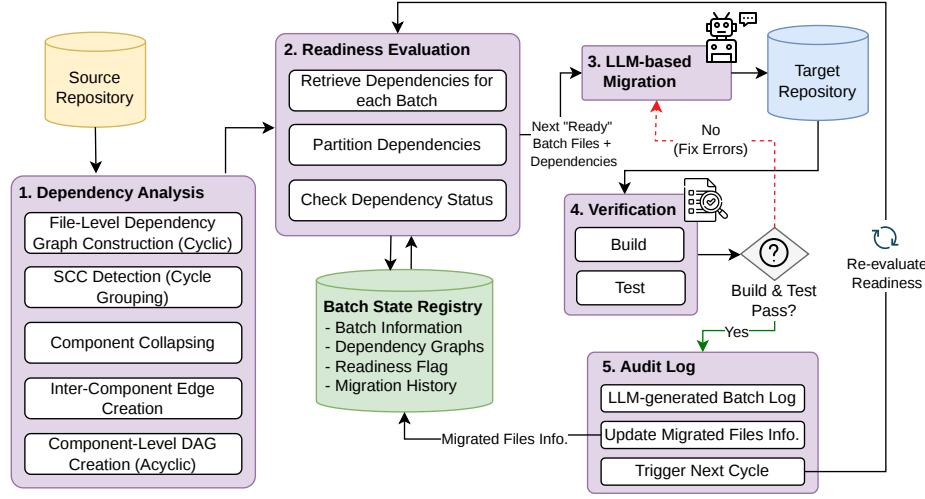


Figure 1: Overview of the dependency-aware migration framework. The process begins with dependency analysis (file-level graph construction and SCC-based cycle handling), followed by readiness evaluation based on dependency status and migration state. Ready batches are translated using an LLM and validated via compilation and testing, with errors iteratively refined through feedback. Sub-steps within each stage are executed in a top-to-bottom order as shown in the figure.

3 Methodology

We present the methodology of **DepWareTrans**, a framework for repository-level migration in interoperable language settings.

3.1 Problem Definition

Let R denote a software repository written in a source programming language L_s , consisting of a set of source files $F = \{f_1, f_2, \dots, f_n\}$, where f denotes an individual file, and n denotes the total number of files present in the repository. The repository is associated with a build system and a test suite T that defines its functional correctness. Our goal is to migrate R to a target language L_t , producing a transformed repository R' , while preserving both compilation and behavioral correctness.

We focus on interoperable language pairs that share a common compilation target or runtime (e.g., JVM or .NET), such as Java–Kotlin, Java–Scala, and C#–F#. In this setting, components written in L_s and L_t can be compiled and executed jointly, enabling incremental migration with intermediate mixed-language states. This enables *incremental migration*, in which only a subset of files is translated at a time, while the remaining system continues to operate in the original language. A key challenge in this setting arises from the *dependency structure* of the repository. Source files are interconnected through type references, method invocations, inheritance relationships, and module-level interactions. Dependencies can be modeled as a directed graph $G = (F, E)$, where nodes correspond to files and edges represent dependency relationships.

Correctness in repository-scale migration is governed not only by the local correctness of individual translations, but by the *consistency of interdependent components*. Translating files in isolation may lead to unresolved symbols, inconsistent type mappings, or incompatible APIs across files, resulting in compilation failures or

incorrect behavior. Therefore, the migration problem can be formulated as identifying an ordering and grouping of files such that translation preserves *dependency consistency*, enabling the system to remain compilable and testable throughout the migration.

In this work, we design a migration approach that (i) preserves compilation correctness at each step, (ii) maintains behavioral correctness with respect to the original test suite T , and (iii) minimizes integration errors arising from cross-file dependencies.

3.2 Overview of the Approach

To address the challenges outlined in Section 3.1, we propose a *dependency-aware migration framework* that aligns the unit of translation with the repository’s dependency structure. Instead of translating files independently, our approach groups interdependent files into *dependency-consistent batches* and performs coordinated translation over these units. Figure 1 presents the end-to-end workflow of the proposed framework. At a high level, the process consists of the following stages:

- (1) **Dependency Analysis:** Construct a file-level dependency graph and identify strongly connected components (SCCs) to capture cyclic relationships.
- (2) **Batch Construction:** Collapse SCCs to form a component-level directed acyclic graph (DAG), which is partitioned into dependency-consistent batches.
- (3) **Readiness Evaluation:** Determine which batches are ready for migration based on dependency readiness and the current migration state.
- (4) **Batch-Level Translation:** Translate each ready batch using an LLM, incorporating context from its dependencies.

Algorithm 1 Dependency-Aware Incremental Repository Migration

Require: Source repository R , source language L_s , target language L_t , test suite T

Ensure: Migrated repository R'

```

1: Extract source files  $F \leftarrow \{f_1, f_2, \dots, f_n\}$  from  $R$ 
2: Construct file-level dependency graph  $G = (F, E)$ 
3: Compute strongly connected components (SCCs) of  $G$ 
4: Collapse SCCs to obtain component-level DAG  $G' = (C, E')$ 
5: Construct dependency-consistent batches  $\mathcal{B} = \{B_1, B_2, \dots, B_m\}$  from  $G'$ 
6: Initialize migrated file set  $M \leftarrow \emptyset$ 
7: Initialize migration state registry  $\mathcal{S}$ 
8: while there exists an unmigrated batch in  $\mathcal{B}$  do
9:    $B \leftarrow \text{SELECTREADYBATCH}(\mathcal{B}, M, \mathcal{S})$ 
10:   $ctx \leftarrow \text{BUILDBATCHCONTEXT}(B, M, R)$ 
11:   $\hat{B} \leftarrow \text{TRANSLATEBATCHWITHLLM}(B, ctx, L_s, L_t)$ 
12:  while true do
13:    Integrate  $\hat{B}$  into partially migrated repository  $R$ 
14:     $buildOK \leftarrow \text{BUILD}(R)$ 
15:    if  $buildOK$  then
16:       $testOK \leftarrow \text{TEST}(R, T)$ 
17:    else
18:       $testOK \leftarrow \text{false}$ 
19:    end if
20:    if  $buildOK \wedge testOK$  then
21:      break
22:    else
23:       $feedback \leftarrow \text{COLLECTERRORSANDFAILURES}(R)$ 
24:       $\hat{B} \leftarrow \text{REFINEBATCHWITHLLM}(\hat{B}, feedback, ctx)$ 
25:    end if
26:  end while
27:   $M \leftarrow M \cup \{\text{files in } B\}$ 
28:  Update migration state registry  $\mathcal{S}$  with migrated files, dependency status, and validation outcomes
29: end while
30:  $R' \leftarrow R$ 
31: return  $R'$ 

```

- (5) **Iterative Validation:** Compile and test the translated code using the original test suite; errors are used to iteratively refine the translation.
- (6) **Incremental Migration:** Repeat the process iteratively, updating migrated files and re-evaluating dependencies to enable subsequent batches while maintaining executability.

By combining dependency-aware batching with iterative validation, the framework ensures that translation units reflect the structural dependencies of the system. This enables consistent integration, reduces dependency-induced failures, and supports reliable incremental migration in real-world repositories.

Algorithm 1 summarizes the overall dependency-aware migration process. Starting from a source repository, the algorithm constructs a file-level dependency graph, collapses cyclic dependencies into SCCs, and forms dependency-consistent batches. Migration then proceeds incrementally by selecting ready batches, translating them with an LLM, and iteratively refining them based on build and test feedback until the entire repository is migrated.

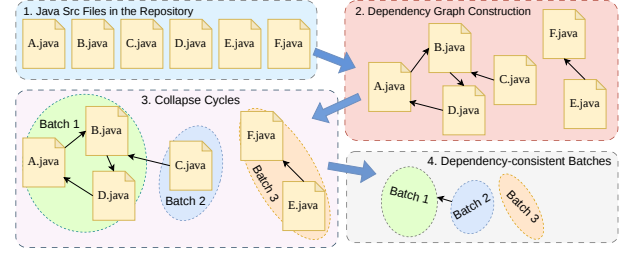


Figure 2: Dependency-aware batch construction. Source files are modeled as a dependency graph; cycles are collapsed into SCCs as atomic units. The graph is then partitioned into dependency-consistent batches, grouping interdependent files for coherent translation.

3.3 Dependency Analysis

Given a repository R with source files $F = \{f_1, f_2, \dots, f_n\}$, we first extract its dependency structure by constructing a directed graph $G = (F, E)$, where each node corresponds to a source file and each edge $(f_i, f_j) \in E$ denotes that file f_i depends on file f_j . Dependencies are identified from inter-file relationships such as type references, inheritance, import statements, and method invocations, which also capture dynamic interactions and exceptional control flows across files.

As illustrated in Figure 2, this dependency graph captures the structural coupling between files and reveals both acyclic and cyclic relationships. Cyclic dependencies arise when groups of files are mutually dependent, which is common in real-world systems due to shared abstractions and bidirectional interactions.

To enable structured reasoning, we compute the SCCs[21] of G . Each SCC represents a maximal set of mutually dependent files and is treated as an atomic unit. We collapse each SCC into a single component node, producing a component-level graph $G' = (C, E')$, where C is the set of components.

By construction, G' is a DAG, as shown in Figure 1, where each node represents a strongly connected component of files. This transformation abstracts cyclic dependencies and provides a component-level dependency structure that serves as the foundation for subsequent batch construction.

3.4 Dependency-Aware Batch Construction

Formally, a *dependency-consistent batch* is a set of files induced by one or more components in the component-level DAG $G' = (C, E')$, such that: (i) mutually dependent files remain within the same batch, and (ii) every dependency leaving the batch is either already resolved by previously migrated batches. Building on the component-level DAG $G' = (C, E')$ obtained from the dependency analysis phase, we construct *dependency-consistent batches* that serve as the unit of translation. Each batch is constructed based on the component structure of G' and ultimately consists of a set of source files. SCCs are treated as atomic units during batching unless further split due to size constraints and are designed to preserve dependency consistency within the translation unit.

The key objective is to group components such that most dependencies are resolved within the batch, thereby minimizing cross-batch dependencies and reducing integration inconsistencies. As illustrated in Figure 2, this results in batches that align with the structural organization of the repository.

Batch Formation. We leverage the acyclic structure of G' to guide batch construction. Since G' admits a topological ordering, we iteratively group components whose dependencies are either: (i) already satisfied by previously migrated batches, or (ii) contained within the current batch. This grouping strategy approximates *dependency closure*, ensuring that interdependent components are translated together.

Handling Cyclic Dependencies. Since cyclic dependencies are resolved during the dependency analysis phase via SCC collapsing, each component in G' already represents a dependency-consistent unit. As a result, mutually dependent files are guaranteed to be included within the same batch, avoiding inconsistencies that arise from independent translation.

Batch Readiness. At each step, we evaluate the *readiness* of a batch based on its dependencies, as illustrated in Figure 1. A batch is considered ready for migration if all its external dependencies have been: (a) migrated in previous steps, or (b) included within the batch itself. This readiness criterion enables incremental migration while preserving system correctness.

Discussion. Unlike prior approaches [13, 27] that operate on individual files or fine-grained fragments, our method constructs translation units corresponding to subgraphs of the dependency graph. This shift from node-level to subgraph-level translation ensures structural consistency across interdependent components and significantly reduces dependency-induced errors during migration.

3.5 Batch Translation and Validation

Once a batch is determined to be ready, we perform *batch-level translation* using a LLM. Unlike file-level approaches, where each file is translated independently, our method translates all files within a batch in a coordinated manner, providing contextual information about related components. The batch-level translation and validation process corresponds to the LLM-based migration and verification stages shown in Figure 1.

Batch-Level Translation. For each batch, we construct an input context that includes: (a) the source code of files within the batch, and (b) relevant dependency information, such as referenced types, method signatures, and imported interfaces. Bounded batch size ensures the input context remains within the model’s input limits.

Iterative Validation. After translating a batch, we validate the result through compilation and test execution, as shown in Figure 1. Specifically, we: (1) attempt to compile the partially migrated repository, and (2) execute the original test suite T to verify behavioral correctness. This is enabled by the interoperability of the source and target languages, which allows translated and untranslated components to be compiled and executed jointly. If compilation or test failures occur, the translation is refined based on the feedback from these errors. This process is repeated iteratively until the batch satisfies compilation and test requirements. Since translation is performed at the batch level, error resolution can account for

interactions across multiple files, enabling more effective fixes than isolated file-level refinement.

3.6 Incremental Migration Workflow

Our framework performs migration in an incremental manner, translating the repository batch by batch while maintaining system executability throughout the process. At each iteration, a ready batch is selected based on dependency constraints (Section 3.4) and translated using the procedure described in Section 3.5. Once validated, the batch is integrated into the target repository, and the migration state is updated. This includes recording migrated components, updating dependency status, and logging translation outcomes, as illustrated in Figure 1.

Interoperability. A key enabler of incremental migration is the interoperability between the source and target languages. Components written in L_s and L_t can coexist and interact, allowing partially migrated systems to compile and execute correctly. This property ensures that validation can be performed continuously during migration.

Migration State Management. We maintain a migration state that tracks: which components have been translated, the dependency status of remaining components, and validation outcomes for each batch. This state is used to guide batch readiness evaluation and to ensure that dependencies are resolved before translation.

Termination. The process continues until all batches have been translated, resulting in a fully migrated repository R' that satisfies compilation and test correctness criteria.

Discussion. By structuring migration as an incremental process, our approach avoids the risks associated with monolithic translation. Instead, it enables continuous validation and controlled integration, making it suitable for large, real-world repositories.

4 Experimental Setup

Subjects. We evaluate our approach on both open-source and industrial repositories. For the Java→Kotlin migration (RQ1 and RQ2), we consider six widely used Apache Commons projects and a large industrial system (STAR). The open-source repositories range from 6K to 29K LOC and exhibit high test coverage (up to 99% line coverage). Detailed statistics, including test coverage, are reported in Table 1. These repositories are widely used in prior software engineering studies as evaluation subjects or benchmarks [12, 13, 30]. For RQ3, we extend the evaluation to additional interoperable language pairs, including Java→Scala and C#→F#. We consider representative repositories such as Commons Text [7], GuardClauses [9], MediatR [17], and csharp-mcp [14], enabling assessment across both ecosystems and models.

Industrial System. STAR is an internal web-based time-tracking and project management system with approximately 400 daily users across multiple organizational roles. The system has been under continuous development since 2018 and supports functionalities such as time entry and approval workflows, leave management, project assignment and effort tracking, and automated billing integration. It includes a role-based access control system and integrates with external accounting systems. The codebase comprises 51K LOC and 1,470 Java files organized into multiple functional modules. Importantly, STAR includes a comprehensive test suite, with test

code exceeding production code in size, enabling robust validation of behavioral correctness during migration.

Baselines and Variants. We compare three configurations:

- **File-by-file:** Each source file is translated independently without feedback.
- **File-by-file + Feedback:** Each file is translated iteratively, incorporating compiler and test feedback, with a maximum of 10 attempts per file.
- **DepWareTrans (ours):** Translates dependency-consistent batches with iterative feedback.

For ablation, we also evaluate **DepWareTrans (w/o Feedback)**, which performs dependency-aware batching without iterative refinement.

Migration Workflow. In the file-by-file setting, each Java file is translated independently while the rest of the repository remains unchanged. After each translation attempt, we compile the entire repository; if compilation succeeds, we execute the test suite. Failures trigger a feedback loop: compiler and test errors are provided to the LLM for refinement. Each file is retried up to 10 iterations.

In our approach, we first construct a dependency graph using static analysis based on imports, type references, and package structure. We then group mutually dependent files into batches (maximum 25 files per batch) using strongly connected components and process batches in a dependency-consistent order. Each batch is translated jointly, compiled, and tested. If failures occur, we iteratively refine the batch using feedback until success.

Metrics. We evaluate translation effectiveness using:

- **Compile Success (%):** Percentage of units (files or batches) that successfully compile.
- **Test Success (%):** Percentage of units that both compile and pass all tests.

Because the migration unit differs across methods (files versus dependency-consistent batches), all methods are compared using the same external criterion: after each migration step, we rebuild the entire partially migrated repository and run the full test suite. Thus, correctness is assessed at the repository state level rather than by local unit validity.

Implementation Details. All Java-based experiments are conducted using Maven (version 3.9.11) with Java 17. Kotlin translation is performed using Kotlin 1.9.25 via Maven plugins. For .NET experiments, we use .NET SDK 10.0.103 with runtime support for .NET 8 and 10. Scala experiments are executed within the Maven toolchain.

We use GPT-5.3 Codex (medium reasoning setting, default parameters) for all experiments. To assess generalizability across models, we additionally evaluate the Java→Scala setting using Claude Opus 4.6 and GPT-5.4. No hyperparameter tuning is performed.

The prompting strategy differs by configuration. In the file-by-file baseline, the model receives a single source file and produces a translated version. In our approach, the model operates on dependency-consistent batches, where the full source code of all files in the batch is included in the prompt, together with selected dependency context (e.g., referenced types, method signatures, and interfaces) from related files, rather than the entire dependency closure. In feedback-based settings, compiler and test errors from

previous attempts are incorporated into the prompt to guide refinement. Full prompting templates and scripts are available in our artifact repository.

Stopping Criteria. In the baseline, each file is retried up to 10 iterations to avoid unbounded execution. In our approach, no fixed iteration limit is imposed; however, we observe that convergence typically occurs within a small number of iterations, and only one case exceeds 10 iterations as shown in Table 4.

5 Results

In this section, we discuss the study results by posing and answering three research questions.

5.1 RQ1: Limitations of File-Level Migration

To investigate how file-level migration strategies perform in real-world scenarios, we evaluate their effectiveness, with and without iterative feedback, across a diverse set of open-source repositories and an industrial system (STAR). The results are summarized in Table 1. We report coverage at the repository level, as tests are defined and executed globally rather than per feature. Since correctness is evaluated based on full-system compilation and test execution, repository-level coverage is the relevant metric.

On smaller open-source repositories, file-by-file translation achieves moderate performance, with compilation success ranging from 50.00% to 86.49% and test success from 41.67% to 81.08%. Incorporating iterative feedback significantly improves results in several cases, achieving 100% compilation and test success for projects such as *commons-cli* and *commons-codec*. However, this improvement is not consistent across all repositories. For instance, on *commons-csv*, performance remains limited (58.33% compile and test success), and on *commons-text*, both approaches fail substantially, with only 29.46% compilation and 8.04% test success even after feedback.

The limitations of file-level approaches become more pronounced in the industrial repository. Across its functional modules, file-by-file translation consistently yields low success rates, with compilation often below 25% and test success frequently near zero. While feedback improves compilation in some cases (e.g., from 25.00% to 96.88% for *wochenarbeitszeit*), it fails to translate into corresponding test success, which remains low (16.41%). Overall, at the repository level, performance improves only marginally from 22.11% to 38.16% in compilation and from 8.50% to 9.39% in test success, indicating that feedback alone is insufficient for large, interdependent systems.

To better understand these failures, we analyze the distribution of error types, as shown in Table 2. We observed that failures in the two most problematic repositories are not random translation mistakes, but are overwhelmingly dependency-linkage failures. In *STAR*, nearly all failed attempts correspond to compile-time reference breakages (e.g., missing symbols and packages), with only a negligible fraction attributable to type or API incompatibilities. In *commons-text*, a similar structural pattern emerges: compile-time linkage failures dominate, while the remaining errors are largely runtime linkage issues (e.g., *ClassNotFoundException*), which still reflect failures at dependency boundaries rather than local translation quality.

This cross-repository alignment makes the underlying failure mode explicit. When grouped by error family, both repositories

Table 1: Performance of file-level migration approaches across repositories.

Repository / Feature	Size		Test Coverage		File-by-file		File-by-file + Feedback	
	LOC	#Java	Line (%)	Branch (%)	Compile (%)	Test (%)	Compile (%)	Test (%)
<i>Open-source repositories</i>								
commons-cli[2]	9,718	36	95.90	94.86	69.44	63.89	100.00	100.00
commons-codec[3]	25,431	87	94.89	90.23	80.46	68.97	100.00	100.00
commons-csv[4]	6,295	12	99.59	97.59	50.00	41.67	58.33	58.33
commons-exec[5]	5,177	37	74.75	57.78	86.49	81.08	100.00	97.30
commons-fileupload[6]	7,094	45	57.20	52.34	80.00	80.00	97.78	97.78
commons-text[7]	29,742	112	99.38	97.82	29.46	8.04	29.46	8.04
<i>Industrial repository (STAR) by functional feature</i>								
urlaub	7,896	213	–	–	19.72	4.23	22.07	4.69
stammdaten	7,023	211	–	–	14.69	1.42	17.06	1.42
leistungsnachweis	5,608	199	–	–	24.62	4.02	24.62	4.02
auftrag	5,400	180	–	–	20.00	1.11	54.44	1.11
wochenarbeitszeit	4,423	128	–	–	25.00	11.72	96.88	16.41
krankheit	4,844	114	–	–	11.40	0.88	11.40	0.88
common	2,283	74	–	–	9.46	8.11	10.81	8.11
persistence	2,400	69	–	–	55.07	55.07	59.42	59.42
zeiterfassung	2,587	68	–	–	17.65	0.00	100.00	0.00
stundenuebersicht	1,716	59	–	–	15.25	1.69	15.25	1.69
mitarbeiterimport	768	27	–	–	11.11	3.70	11.11	3.70
populators	1,601	27	–	–	48.15	48.15	48.15	48.15
datapopulator	1,693	26	–	–	65.38	57.69	69.23	61.54
config	1,196	21	–	–	57.14	38.10	57.14	38.10
mitarbeiter	501	15	–	–	13.33	0.00	13.33	0.00
abwesenheitsuebersicht	263	12	–	–	8.33	0.00	100.00	16.67
nonprod-root	406	11	–	–	18.18	18.18	18.18	18.18
dbcleanup	219	10	–	–	20.00	0.00	20.00	0.00
cloudwatch	180	5	–	–	60.00	40.00	60.00	40.00
backend-root	13	1	–	–	100.00	100.00	100.00	100.00
STAR (overall)	51,020	1,470	96.47	93.44	22.11	8.50	38.16	9.39

Table 2: Shared diagnostic signatures across failure-prone repositories (post-feedback).

Error Signature	STAR		commons-text	
	Count	Share (%)	Count	Share (%)
cannot find symbol [24]	12,994	96.98	720	69.90
package ... does not exist [24]	357	2.66	70	6.80
ClassNotFoundException [23]	0	0.00	240	23.30
incompatible types [24]	27	0.20	0	0.00
method ... cannot be applied [24]	1	0.01	0	0.00
Generic test failure (Tests run ...) [8]	10	0.07	0	0.00

concentrate in the same dependency-related classes, indicating a shared structural limitation. Under file-by-file migration, translating a file while its dependency neighborhood remains unmigrated creates unstable mixed-language boundaries, leading to cascading unresolved references and classpath inconsistencies.

These observations suggest that the primary limitation of baseline approaches is not the translation capability of the model, but the

choice of migration unit, which ignores the dependency structure of the repository. This motivates a shift from file-level translation to dependency-aware units (e.g., SCC- or component-level batches), which co-migrate interdependent files and directly target the dominant source of failure identified in RQ1.

Answer to RQ1: File-level migration does not scale to dependency-rich repositories; even with iterative feedback, cross-file dependency inconsistencies remain unresolved, resulting in low end-to-end correctness.

5.2 RQ2: Effectiveness of DepWareTrans

Motivated by the limitations identified in RQ1, where file-level approaches fail in dependency-rich repositories, we evaluate the effectiveness of our proposed dependency-aware migration framework. Specifically, we compare three configurations: (i) file-by-file migration with feedback, (ii) dependency-aware batching without feedback, and (iii) the full approach combining batching with

Table 3: Effectiveness of dependency-aware batching and feedback on repositories where file-level approaches underperform.

Repository	File-by-file + Feedback		DepWareTrans (w/o Feedback)		DepWareTrans		
	Compile (%)	Test (%)	Compile (%)	Test (%)	Compile (%)	Test (%)	Avg. Iter.
Commons-csv[4]	58.33	58.33	66.66	66.66	91.67	91.67	0.80
Commons-text[7]	29.46	8.04	100.00	100.00	100.00	100.00	0.00
STAR	38.16	9.39	7.84	7.84	100.00	100.00	3.45

Table 4: STAR cumulative success rate by feedback iteration budget k .

k	0	1	2	3	4	5	6	7	8	20
Compile Success (%)	7.84	33.33	62.75	88.24	96.08	100.00	100.00	100.00	100.00	100.00
Compile+Test Success (%)	7.84	15.69	31.37	49.02	66.67	82.35	84.31	96.08	98.04	100.00

iterative feedback. The results are summarized in Table 3, with convergence behavior further analyzed in Table 4.

We focus on repositories where file-by-file approaches underperform, as these cases expose the limitations of file-level migration. Repositories where file-level methods already achieve near-perfect results (e.g., commons-cli, commons-codec) provide limited insight into failure modes and are therefore not the focus of this analysis.

Across all repositories, we observe that dependency-aware batching significantly improves migration outcomes compared to file-level workflows. For example, *commons-text*, batching alone is sufficient to fully resolve migration failures, achieving 100% compilation and test success without any feedback, compared to only 29.46% and 8.04% under file-by-file migration with feedback. Consistent with the findings in RQ1, this indicates that failures in this repository are predominantly structural, arising from inconsistent handling of interdependent components. Once dependency consistency is enforced through batching, no further refinement is required. In contrast, *commons-csv* exhibits a mixed failure mode. Batching without feedback improves performance over the file-level baseline (66.66% vs. 58.33%), and the full approach further increases success to 91.67%. However, unlike *commons-text*, batching combined with feedback does not achieve complete success. Manual inspection shows that the remaining failure is concentrated in a single large, central class, approximately 3K LOC, with complex builder logic and strict API/ABI expectations used widely across the repository. This case suggests that, while dependency-aware batching resolves most repository-level inconsistencies, some architectural hub classes require more than iterative LLM-based repair. In particular, large, highly coupled classes may need complementary decomposition, refactoring, or manual guidance before they can be migrated reliably.

For the industrial repository *STAR*, batching alone is insufficient, achieving only 7.84% success, equivalent to the zero-iteration baseline. This does not contradict the RQ1 finding that *STAR* failures are primarily dependency-related: the file-level baseline in RQ1 also used iterative feedback, yet still achieved only 38.16% compilation and 9.39% test success. Rather, the result shows that feedback is ineffective when applied to isolated files under inconsistent dependency boundaries. Dependency-aware batching first establishes a structurally consistent migration unit; once this consistency is in

place, feedback can effectively resolve the remaining translation and integration errors that arise at scale.

When combined with iterative feedback, the full approach achieves 100% compilation and test success, substantially outperforming the file-level baseline. Table 4 further illustrates this convergence behavior. Starting from a low baseline of 7.84% at $k = 0$, performance improves steadily with additional feedback iterations, reaching 49.02% at $k = 3$, 82.35% at $k = 5$, and 98.04% by $k = 8$. Notably, most improvements occur within the first few iterations, suggesting that once dependency consistency is established, remaining errors become localized and can be corrected efficiently through feedback.

Overall, these results demonstrate that batching and feedback address complementary aspects of the migration problem. Batching resolves structural inconsistencies caused by interdependent components, while iterative feedback corrects residual translation and integration errors. However, the results on *commons-csv* also indicate that extremely large or highly central components may require additional handling beyond the current approach.

Answer to RQ2: Dependency-aware batching preserves dependency coherence, while iterative feedback fixes residual errors; together, they enable reliable end-to-end migration.

5.3 RQ3: Generalization Across Language Pairs

We evaluate whether the effectiveness of our approach generalizes beyond Java–Kotlin to other interoperable language ecosystems and across different underlying LLMs. Table 5 summarizes the results across multiple repositories, language pairs, and models.

We first examine the behavior of file-by-file migration with feedback across language pairs. The results show substantial variability in performance depending on the repository and language ecosystem. For example, file-level migration achieves 100% success in *GuardClauses* (C#→F#), but drops to 66.67% compile and 33.33% test success in *csharp-mcp*, and further to 29.46% compile and 8.04% test success in *Commons Text* (Java→Scala). This variability indicates that file-by-file workflows are highly sensitive to repository structure and dependency complexity, and do not generalize reliably across language pairs.

Table 5: Generalization across interoperable language pairs, comparing file-by-file + feedback with DepWareTrans variants.

Repository	Source→ Target	LLM Used	File-by-file + Feedback		DepWareTrans (w/o Feedback)		DepWareTrans		
			Compile (%)	Test (%)	Compile (%)	Test (%)	Compile (%)	Test (%)	Avg. Iter.
GuardClauses[9]	C#→F#	Codex 5.3	100.00	100.00	40.00	40.00	100.00	100.00	2.60
MediatR[17]	C#→F#	Codex 5.3	41.86	41.86	100.00	66.67	100.00	100.00	0.00
csharp-mcp[14]	C#→F#	Codex 5.3	66.67	33.33	0.00	0.00	100.00	100.00	1.00
Commons Text[7]	Java→Scala	Codex 5.3	29.46	8.04	78.56	59.82	100.00	100.00	0.68
Commons Text[7]	Java→Scala	Claude Opus 4.6	26.78	8.04	47.37	31.58	100.00	100.00	0.89
Commons Text[7]	Java→Scala	GPT-5.4	24.10	7.14	5.88	5.88	100.00	100.00	5.59

Dependency-aware batching without feedback provides more consistent improvements, but its effectiveness remains limited and repository-dependent. For instance, batching alone achieves 40.00% success in *GuardClauses*, 47.37%/31.58% in *Commons Text*, and fails entirely in *csharp-mcp* (0.00%). These results are consistent with RQ2, suggesting that while batching resolves structural inconsistencies, it does not address residual translation errors or complex semantic interactions. In contrast, the full approach combining batching with iterative feedback consistently achieves 100% compilation and test success across all repositories, language pairs, and models. This includes both JVM-based (Java→Scala) and .NET-based (C#→F#) ecosystems, as well as different LLMs (Codex 5.3, GPT-5.4 and Claude Opus 4.6). Notably, the approach achieves these results even when file-level workflows fail significantly, as in *Commons Text* and *csharp-mcp*. These results highlight two key aspects of generalization. First, the limitations of file-by-file migration are consistent across language pairs: performance degrades in repositories with complex dependency structures, regardless of the specific languages involved. Second, the improvements introduced by DepWareTrans are stable across both ecosystems and models, indicating that the gains arise from the migration strategy, specifically, enforcing dependency consistency and iterative refinement rather than model-specific capabilities.

Overall, these findings suggest that dependency-aware batching combined with feedback provides a robust and generalizable solution for repository-level migration in interoperable language settings, while file-level approaches remain brittle and highly sensitive to repository structure.

Answer to RQ3: Across the evaluated interoperable language pairs and LLMs, DepWareTrans generalizes effectively: file-by-file migration shows unstable performance, whereas dependency-aware batching with iterative feedback consistently achieves full correctness in our experiments.

6 Discussion

Why Dependency-Aware Migration Works. The results across RQ1–RQ3 indicate that dependency inconsistency is the dominant bottleneck in repository-level code translation.

Problem. File-level translation violates implicit invariants across interdependent components. In large repositories, classes are tightly

coupled through shared data models, inheritance, and APIs. Translating such components independently produces inconsistent intermediate states, leading to unresolved references, type mismatches, and linkage errors. As a result, iterative feedback alone is insufficient: it can correct local errors, but cannot restore consistency across independently translated components.

Solution. Dependency-aware batching addresses this by enforcing *local consistency closure*. By co-migrating interdependent files, it ensures that symbols, types, and interfaces remain consistent within the translation unit. This explains the behavior observed in *commons-text*, where failures are primarily structural and are fully resolved by batching alone. However, structural consistency is not sufficient in all cases. In the industrial repository *STAR*, batching must be combined with iterative feedback to achieve full correctness. This indicates that, once dependency inconsistencies are removed, remaining errors stem from translation-level issues such as control flow, API usage, and language-specific constructs. Feedback complements batching by refining these aspects. The results for *commons-csv* highlight a limitation of the approach. While batching and feedback improve success rates, they struggle with large, central components that act as architectural hubs. In such cases, translation errors propagate widely due to strong coupling and strict API constraints. Addressing these scenarios may require additional techniques, such as specialized handling of large classes or interface-driven translation.

Overall, repository-level migration can be viewed as preserving cross-component invariants under dependency constraints. Dependency-aware batching addresses the structural dimension of this problem, while iterative feedback resolves residual errors, together enabling reliable migration in practice.

Resource and Cost Analysis. Across the three repositories shared by the file-by-file baseline and DepWareTrans (Table. 3), the clearest resource difference appears in total token consumption rather than overall active time. The file-by-file baseline consumed 16,941,405 total tokens, whereas DepWareTrans consumed 4,990,616, corresponding to a 70.5% reduction overall. This reduction is consistent across all three repositories: *commons-csv* drops from 1,047,288 to 97,848 total tokens, *commons-text* from 5,442,491 to 555,712, and *STAR* from 10,451,626 to 4,337,056. In contrast, total active time across the same repositories is similar overall, with file-by-file requiring about 21h 59m 32s and DepWareTrans 21h 52m 16s; however, the per-repository pattern differs, as DepWareTrans

is slower on the smaller OSS repositories (commons-csv: 7m 59s vs. 3m 19s; commons-text: 1h 42m 21s vs. 34m 21s) but modestly faster on STAR (20h 01m 56s vs. 21h 21m 51s). Overall, these results suggest that DepWareTrans’s primary efficiency advantage lies in substantially reducing token consumption, while its time advantage becomes more visible on larger, dependency-heavy systems.

Implications for Practitioners. Our results have direct implications for practitioners performing large-scale language migration.

File-by-file migration, even when supported by modern LLMs and feedback loops, is insufficient for systems with non-trivial dependency structure. As demonstrated in the industrial case (Table 1), such approaches yield low compilation and test success rates despite multiple refinement attempts.

Dependency-aware batching provides a practical strategy for incremental migration. By ensuring that interdependent files are migrated together, the approach avoids unstable mixed-language boundaries that lead to cascading failures. Combined with iterative feedback, this enables reliable end-to-end migration while preserving the ability to validate intermediate states through compilation and testing. The iteration results (Table 4) suggest that the cost of refinement is manageable in practice. Once dependency consistency is established, most errors are resolved in a few iterations, enabling feasible real-world deployment.

Lessons from the Industrial Case. Our experience with the STAR repository provides several practical insights. First, feedback alone is insufficient at the file level, as it cannot resolve failures caused by inconsistent dependency boundaries. This is evident from the limited improvement between file-by-file and file-by-file with feedback in RQ1. Dependency-aware batching significantly reduces failure modes, but does not completely eliminate the need for refinement in complex systems. While batching alone achieves full success in simpler repositories, the industrial setting requires iterative feedback to resolve residual translation and integration issues. Once dependency consistency is enforced, convergence is rapid. As shown in Table 4, success rates increase sharply within the first few iterations, indicating that most remaining errors are localized and can be corrected efficiently.

Applicability and Scope. The effectiveness of our approach depends on several key conditions. First, it is only well-suited to interoperable or co-executable language pairs (e.g., JVM- or .NET-based ecosystems), where mixed-language execution allows incremental migration. Second, the approach assumes the availability of a buildable system and an executable test suite, which are necessary for feedback-driven refinement. The approach may be less effective in settings where dependencies cannot be reliably extracted, or where test coverage is insufficient to validate correctness. Additionally, for non-interoperable language pairs, where incremental execution is not feasible, alternative migration strategies may be required.

Implications for Research. Our findings suggest that improving repository-level code translation requires reconsidering both evaluation and methodology. Current approaches that focus on file- or function-level benchmarks do not capture the structural challenges observed in real-world systems. As shown in this work, dependency-aware migration provides a more appropriate abstraction for large-scale translation tasks. Furthermore, the results indicate that structural factors play a critical role in translation success, independent of the underlying LLM. This suggests that future work

should focus not only on improving model capabilities but also on designing translation frameworks that explicitly account for dependency structure and system-level consistency.

7 Threats to Validity

Internal Validity. This concerns whether observed improvements are attributable to our approach. We mitigate LLM non-determinism by using fixed prompts and default parameters across all experiments, without task-specific tuning, and apply the feedback loop consistently across configurations to ensure fair comparison. Our approach also relies on static dependency extraction (imports, type references, package structure), which may introduce inaccuracies. However, consistent results across repositories suggest robustness to minor inference errors.

Construct Validity. We measure effectiveness using compilation and test success rates, where a unit is considered successful only if it both compiles and passes all tests. While these metrics reflect practical correctness, they depend on the quality and completeness of the test suites; limited coverage may not fully guarantee semantic equivalence.

External Validity. This concerns the generalizability of our findings. We evaluate our approach on multiple open-source repositories and a large industrial system, across several interoperable language pairs (Java–Kotlin, Java–Scala, and C#–F#). However, results may not generalize to non-interoperable language pairs, where incremental migration and mixed-language execution are not feasible. Moreover, the effectiveness of our approach depends on the availability of buildable systems and executable test suites, which may not exist in all real-world repositories. The industrial case study focuses on a single organization and domain, which may limit its generalizability to other contexts.

Conclusion Validity. The consistently large performance differences between baseline and proposed approaches reduce the likelihood that improvements are due to random variation. However, the limited number of repositories evaluated in certain settings (e.g., RQ2 and RQ3) may affect the generality of the conclusions, and further studies would strengthen confidence.

8 Conclusion

This paper revisits repository-level migration as a problem of structural consistency in interoperable language settings. While recent advances in LLMs have improved local code translation, our results show that these gains do not translate to end-to-end correctness when migrating real-world systems incrementally. Across both open-source and industrial repositories, failures primarily arise from inconsistencies at dependency boundaries rather than errors in individual file translations.

We propose **DepWareTrans**, a dependency-aware migration framework that elevates the unit of translation from isolated files to dependency-consistent batches, combined with iterative compile-and-test-driven validation. Our evaluation demonstrates that this approach enables reliable, end-to-end migration in settings where file-level workflows fail, and that the benefits generalize across multiple interoperable language pairs.

These findings suggest that effective repository-level migration in co-executable settings must explicitly incorporate dependency

structure into both migration units and validation. This perspective opens new directions for structure-aware transformation, including improved handling of architectural hotspots and tighter integration between dependency analysis and LLM-based translation.

Data Availability Statement

We release a replication package with scripts, configurations, and open-source repositories, available at <https://doi.org/10.6084/m9.figshare.32133676>. While we provide STAR's migration scripts and results, confidentiality constraints prevent releasing its source code. Access to the STAR codebase may be granted upon reasonable request, subject to organizational approval.

References

- [1] Android Developers. 2024. *Android's Kotlin-first approach*. <https://developer.android.com/kotlin/first> Accessed: 2026-04-07.
- [2] Apache Commons CLI Project. 2026. Apache Commons CLI. <https://github.com/apache/commons-cli>. Accessed: 2026-04-13.
- [3] Apache Commons Codec Project. 2026. Apache Commons Codec. <https://github.com/apache/commons-codec>. Accessed: 2026-04-13.
- [4] Apache Commons CSV Project. 2026. Apache Commons CSV. <https://github.com/apache/commons-csv>. Accessed: 2026-04-13.
- [5] Apache Commons Exec Project. 2026. Apache Commons Exec. <https://github.com/apache/commons-exec>. Accessed: 2026-04-13.
- [6] Apache Commons FileUpload Project. 2026. Apache Commons FileUpload. <https://github.com/apache/commons-fileupload>. Accessed: 2026-04-13.
- [7] Apache Commons Text Project. 2026. Apache Commons Text. <https://github.com/apache/commons-text>. Accessed: 2026-04-13.
- [8] Apache Maven Project. 2026. Maven Surefire Plugin: Error Summary. <https://maven.apache.org/surefire/maven-surefire-plugin/newerrorssummary.html> Accessed: 2026-04-09.
- [9] Ardalis. 2026. GuardClauses: A Simple Package with Guard Clause Extensions. <https://github.com/ardalis/GuardClauses>. Accessed: 2026-04-13.
- [10] Peng Di, Jianguo Li, Hang Yu, Wei Jiang, Wenting Cai, Yang Cao, Chaoyu Chen, Dajun Chen, Hongwei Chen, Liang Chen, Gang Fan, Jie Gong, Zi Gong, Wen Hu, Tingting Guo, Zhichao Lei, Ting Li, Zheng Li, Ming Liang, Cong Liao, Bingchang Liu, Jiachen Liu, Zhiwei Liu, Shaojun Lu, Min Shen, Guangpei Wang, Huan Wang, Zhi Wang, Zhaogui Xu, Jiawei Yang, Qing Ye, Gehao Zhang, Yu Zhang, Zelin Zhao, Xunjin Zheng, Hailian Zhou, Lifu Zhu, and Xianying Zhu. 2024. CodeFuse-13B: A Pretrained Multi-lingual Code Large Language Model. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. ACM, New York, NY, USA, 418–429. doi:10.1145/3639477.3639719
- [11] Qiong Feng, Huan Ji, Xiaotian Ma, and Peng Liang. 2024. Cross-Language Dependencies: An Empirical Study of Kotlin-Java. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (Barcelona, Spain) (ESEM '24)*. Association for Computing Machinery, New York, NY, USA, 189–199. doi:10.1145/3674805.3686680
- [12] Michael Hilton, Jonathan Bell, and Darko Marinov. 2018. A large-scale study of test coverage evolution. In *ASE '18: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (Montpellier, France) (ASE '18)*. Association for Computing Machinery, New York, NY, USA, 53–63. doi:10.1145/3238147.3238183
- [13] Ali Reza Ibrahimzade, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. 2025. AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025). doi:10.1145/3729379
- [14] InfinityFlowApp. 2026. csharp-mcp: C# MCP Framework. <https://github.com/InfinityFlowApp/csharp-mcp>. Accessed: 2026-04-13.
- [15] JetBrains. 2026. *Java to Kotlin Conversion Comes to Visual Studio Code*. <https://blog.jetbrains.com/kotlin/2026/02/java-to-kotlin-conversion-comes-to-visual-studio-code/> Accessed: 2026-04-07.
- [16] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the Evaluation of Neural Code Translation: Taxonomy and Benchmark. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Press, 1529–1541. doi:10.1109/ASE56229.2023.00114
- [17] Lucky Penny Software. 2026. MediatR: Simple Mediator Implementation in .NET. <https://github.com/LuckyPennySoftware/MediatR>. Accessed: 2026-04-13.
- [18] Jocelyn Luizzi, Jingbo Yang, and Eve Matthae. 2024. *Translating Java to Kotlin at Scale*. <https://engineering.fb.com/2024/12/18/android/translating-java-to-kotlin-at-scale/> Engineering at Meta.
- [19] Matias Martinez and Bruno Gois Mateus. 2022. Why Did Developers Migrate Android Applications From Java to Kotlin? . *IEEE Transactions on Software Engineering* 48, 11 (Nov. 2022), 4521–4534. doi:10.1109/TSE.2021.3120367
- [20] Bruno Gois Mateus, Matias Martinez, and Christophe Kolski. 2023. Learning migration models for supporting incremental language migrations of software applications. *Information and Software Technology* 153 (2023), 107082. doi:10.1016/j.infsof.2022.107082
- [21] Vlad-Andrei Munteanu. 2018. Strongly connected components-Algorithm for finding the strongly connected components of a graph. *arXiv preprint arXiv:1802.05387* (2018). doi:10.48550/arXiv.1802.05387
- [22] Vikram Nitin, Rahul Krishna, and Baishakhi Ray. 2024. SpecTra: Enhancing the Code Translation Ability of Language Models by Generating Multi-Modal Specifications. *arXiv preprint arXiv:2405.18574* (2024). <https://arxiv.org/abs/2405.18574>
- [23] Oracle. 2026. ClassNotFoundException (Java SE API). <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/ClassNotFoundException.html> Accessed: 2026-04-09.
- [24] Oracle 2026. *Javac: The Java Compiler*. Oracle. <https://docs.oracle.com/en/java/javase/17/docs/specs/man/javac.html> Accessed: 2026-04-09.
- [25] Rangeet Pan, Ali Reza Ibrahimzade, Rahul Krishna, Divya Sankar, Lambert Pougum Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *ICSE '24: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal)*. Article 82, 13 pages. doi:10.1145/3597503.3639226
- [26] Manish Shetty, Naman Jain, Adwait Godbole, Sanjit A Seshia, and Koushik Sen. 2024. Syzygy: Dual code-test c to (safe) rust translation using llms and dynamic analysis. *arXiv preprint arXiv:2412.14234* (2024). doi:10.48550/arXiv.2412.14234
- [27] Yanli Wang, Yanlin Wang, Suiquan Wang, Daya Guo, Jiachi Chen, John Grundy, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, and Zibin Zheng. 2026. RepoTransBench: A Real-World Multilingual Benchmark for Repository-Level Code Translation. *IEEE Transactions on Software Engineering* 52, 2 (2026), 675–690. doi:10.1109/TSE.2025.3645056
- [28] Weixiang Yan, Yuchen Tian, Yunzhe Li, Qian Chen, and Wen Wang. 2023. CodeTransOcean: A Comprehensive Multilingual Benchmark for Code Translation. In *Findings of the Association for Computational Linguistics: EMNLP*. 5067–5089. doi:10.18653/v1/2023.findings-emnlp.337
- [29] Aidan ZH Yang, Yoshiki Takashima, Brandon Paulsen, Josiah Dodds, and Daniel Kroening. 2024. VERT: Verified Equivalent Rust Transpilation with Large Language Models as Few-Shot Learners. *arXiv preprint arXiv:2404.18852* (2024). <https://arxiv.org/abs/2404.18852>
- [30] Lin Yang, Junjie Chen, Hanmo You, Jiachen Han, Jiajun Jiang, Zhe Sun, Xinqi Lin, Fang Liang, and Yuning Kang. 2023. Can Code Representation Boost IR-Based Test Case Prioritization?. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. 240–251. doi:10.1109/ISSRE59848.2023.00077
- [31] Xin Yin, Chao Ni, Tien N Nguyen, Shaohua Wang, and Xiaohu Yang. 2024. Rectifier: Code translation with corrector via llms. *arXiv preprint arXiv:2407.07472* (2024). doi:10.48550/arXiv.2407.07472
- [32] Hanliang Zhang, Cristina David, Meng Wang, Brandon Paulsen, and Daniel Kroening. 2025. Scalable, Validated Code Translation of Entire Projects using Large Language Models. *Proc. ACM Program. Lang.* 9, PLDI, Article 212 (June 2025), 26 pages. doi:10.1145/3729315

Received 2026-04-30; accepted 2026-07-01